

# **SIMATS ENGINEERING ASSESSMENT TOOL 3**

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**

**COURSE CODE: CSA1108**

---

## **COURSE OUTCOME COVERED**

**CO1:** *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

*Assessment Tool Weightage: 30%*

---

Dr. A. MANIMARAN

**QUESTIONS: 6**

**Total Marks:30**

Annexure A

## **SHORT ANSWER TEST**

---

### **Code of Conduct:**

*I E.Ragul / 192571032(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

*E. Ragul*

## Annexure A

### Short Answer Test

1. Suppose a software system is developed for a Smart Banking System where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- **Class**
- **Object**
- **Message Passing**
- **Encapsulation**
- **Polymorphism**

(b) Describe the following object relationships:

- **Association**
- **Aggregation**
- **Composition**

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

**Answer:**

(a) Object-Oriented Concepts

| Concept                | Explanation                                                                | Example (Smart Banking System)                                                                    |
|------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Class</b>           | A class is a blueprint or template for creating objects.                   | Customer, Account, Transaction, Bank                                                              |
| <b>Object</b>          | An object is an instance of a class with its own data and behavior.        | Customer John with Account No. 1001                                                               |
| <b>Message Passing</b> | Objects communicate by sending messages (calling methods).                 | Customer.transferFunds() sends a request to the Account object.                                   |
| <b>Encapsulation</b>   | Combining data and methods into a single unit and hiding internal details. | Account balance is private and can be accessed only through methods like deposit() or withdraw(). |
| <b>Polymorphism</b>    | The same method performs different actions depending on the object.        | makePayment() works differently for electricity, water, and internet bill payments.               |

### (b) Object Relationships

| Relationship | Description                                                          | Example (Smart Banking System)                                                                                       |
|--------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Association  | A general relationship where two classes interact.                   | A <b>Customer</b> owns one or more <b>Accounts</b> .                                                                 |
| Aggregation  | A "has-a" relationship where objects can exist independently.        | A <b>Bank</b> has many <b>Customers</b> . Customers can still exist even if a bank branch is closed.                 |
| Composition  | A strong "part-of" relationship where one object depends on another. | An <b>Account</b> contains <b>Transaction</b> records. If the account is deleted, its transactions are also removed. |

### (c) How OOP Concepts Improve Reusability and Maintainability

| Concept                                | Benefit in Banking System                                                                                 |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Class & Object                         | Makes the system modular by organizing related data and functions.                                        |
| Encapsulation                          | Protects sensitive information like account balance and customer details.                                 |
| Polymorphism                           | Allows adding new payment methods or account types without changing existing code.                        |
| Association, Aggregation & Composition | Clearly define relationships between banking entities, making the system easier to understand and modify. |
| Code Reusability                       | Existing classes (Customer, Account, Transaction) can be reused in new banking features.                  |
| Maintainability                        | Changes in one module have minimal impact on other modules, making updates and debugging easier.          |

### Final Answer:

The **Smart Banking System** applies **Object-Oriented Programming (OOP)** concepts such as **classes, objects, encapsulation, message passing, and polymorphism** to create a structured and modular design. Relationships like **association, aggregation, and composition** organize interactions between banking entities. These concepts improve **software reusability, security, scalability, and maintainability**, making the banking system easier to develop, test, and enhance in the future.

## 2. Consider developing a Smart Healthcare Management System.

(a) Explain the following object-oriented design principles:

- **Abstraction**
- **Encapsulation**
- **Modularity**

(b) Describe the following SOLID principles:

- **Liskov Substitution Principle (LSP)**
- **Interface Segregation Principle (ISP)**

(c) Explain how these principles improve software quality and flexibility.

**Answer:**

### a) Object-Oriented Design Principles

| Principle            | Explanation                                                                             | Example (Smart Healthcare Management System)                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Abstraction</b>   | Hides unnecessary implementation details and shows only essential features.             | A <b>Patient</b> can book an appointment without knowing how the appointment is processed internally.                        |
| <b>Encapsulation</b> | Combines data and methods into one unit and protects data from unauthorized access.     | Patient medical records are private and can be accessed only through authorized methods like viewRecord() or updateRecord(). |
| <b>Modularity</b>    | Divides the system into independent modules, making development and maintenance easier. | Separate modules for <b>Patient Management, Doctor Management, Appointment Scheduling, and Billing.</b>                      |

### (b) SOLID Principles

| Principle                                    | Description                                                                                                                          | Example (Smart Healthcare Management System)                                                                                                         |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Liskov Substitution Principle (LSP)</b>   | A subclass should be able to replace its parent class without affecting the program's correctness.                                   | GeneralDoctor and SpecialistDoctor can both replace the Doctor class wherever a doctor object is required.                                           |
| <b>Interface Segregation Principle (ISP)</b> | A class should not be forced to implement methods it does not need. Use smaller, specific interfaces instead of one large interface. | Create separate interfaces such as AppointmentService, PrescriptionService, and BillingService so each class implements only the required functions. |

**(c) How These Principles Improve Software Quality and Flexibility**

| Principle       | Benefit in Healthcare System                                                                |
|-----------------|---------------------------------------------------------------------------------------------|
| Abstraction     | Simplifies system usage by hiding complex implementation details.                           |
| Encapsulation   | Protects sensitive patient information and improves data security.                          |
| Modularity      | Makes the system easier to develop, test, maintain, and upgrade.                            |
| LSP             | Allows new doctor types or healthcare services to be added without changing existing code.  |
| ISP             | Reduces unnecessary dependencies and makes classes simpler and more reusable.               |
| Overall Benefit | Improves software quality, flexibility, scalability, maintainability, and code reusability. |

**Final Answer:**

The **Smart Healthcare Management System** applies **Abstraction, Encapsulation, and Modularity** to build a secure and well-structured application. It also follows the **Liskov Substitution Principle (LSP)** and **Interface Segregation Principle (ISP)** to create flexible, reusable, and maintainable software. These object-oriented principles improve **software quality, scalability, security, and ease of maintenance**, making the healthcare system reliable and adaptable to future requirements.

3. Suppose a software company is developing an Online Food Delivery System.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing
- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

**Answer:**

(a) Phases of the Software Development Life Cycle (SDLC)

| SDLC Phase                  | Explanation                                                                    | Example (Online Food Delivery System)                                                                        |
|-----------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Planning</b>             | Identify project goals, scope, budget, resources, and schedule.                | Decide to develop an app for ordering food online with delivery tracking.                                    |
| <b>Requirement Analysis</b> | Gather and analyze user and business requirements.                             | Customers should be able to register, browse restaurants, place orders, make payments, and track deliveries. |
| <b>Design</b>               | Create the system architecture, database, user interface, and software design. | Design the app screens, database for users, restaurants, orders, and payment system.                         |
| <b>Development</b>          | Write the program code based on the design specifications.                     | Develop modules such as user login, food ordering, payment gateway, and order tracking.                      |
| <b>Testing</b>              | Test the software to identify and fix errors before release.                   | Verify that users can place orders, make payments, and receive correct order updates.                        |
| <b>Maintenance</b>          | Update the software after deployment by fixing bugs and adding new features.   | Fix reported issues, improve performance, and introduce features like coupons or live chat support.          |

(b) Purpose of Each SDLC Phase

| SDLC Phase      | Purpose                                                        |
|-----------------|----------------------------------------------------------------|
| <b>Planning</b> | Defines project objectives, budget, timeline, and feasibility. |

| SDLC Phase                  | Purpose                                                                |
|-----------------------------|------------------------------------------------------------------------|
| <b>Requirement Analysis</b> | Collects and documents user and business requirements accurately.      |
| <b>Design</b>               | Provides a blueprint for developing the software system.               |
| <b>Development</b>          | Converts the design into a working software application.               |
| <b>Testing</b>              | Ensures the software is free from defects and meets user requirements. |
| <b>Maintenance</b>          | Keeps the software updated, secure, and efficient after deployment.    |

### (c) How SDLC Contributes to Delivering Reliable Software

| SDLC Benefit           | Contribution                                                         |
|------------------------|----------------------------------------------------------------------|
| Better Planning        | Reduces project risks and improves resource management.              |
| Clear Requirements     | Ensures the software meets customer needs.                           |
| Structured Design      | Produces a modular and organized system.                             |
| Thorough Testing       | Detects and removes bugs before deployment.                          |
| Continuous Maintenance | Improves software quality by fixing issues and adding enhancements.  |
| Overall Quality        | Delivers reliable, secure, maintainable, and user-friendly software. |

### Final Answer

The **Software Development Life Cycle (SDLC)** provides a systematic approach to developing an **Online Food Delivery System** through the phases of **Planning, Requirement Analysis, Design, Development, Testing, and Maintenance**. Each phase has a specific purpose, ensuring the software is well-planned, correctly implemented, thoroughly tested, and continuously maintained. As a result, SDLC helps deliver **reliable, high-quality, secure, and maintainable software** that satisfies user requirements.

**4. Consider a project for developing a Smart Agriculture Monitoring System, where customer requirements change regularly.**

**(a) Explain the following SDLC models:**

- **Prototype Model**
- **Spiral Model**
- **Agile Model**
- **DevOps Model**

**(b) Compare the Prototype Model and Agile Model.**

**(c) Recommend the most suitable SDLC model for this project with justification.**

**Answer:**

**(a) SDLC Models**

| <b>SDLC Model</b>      | <b>Explanation</b>                                                                                                                           | <b>Example (Smart Agriculture Monitoring System)</b>                                                                               |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prototype Model</b> | A working prototype is developed first to understand customer requirements. Feedback is used to improve the system before final development. | Build a prototype showing crop monitoring and sensor data to gather farmer feedback.                                               |
| <b>Spiral Model</b>    | A risk-driven model where development is completed in repeated cycles of planning, risk analysis, development, and evaluation.               | Each cycle adds features such as soil monitoring, weather forecasting, and irrigation control while reducing project risks.        |
| <b>Agile Model</b>     | Software is developed in small iterations with continuous customer feedback and frequent updates.                                            | New features like pest detection or fertilizer recommendations are added in short development cycles based on farmer requirements. |
| <b>DevOps Model</b>    | Integrates development and operations to enable continuous development, testing, deployment, and monitoring.                                 | The system is continuously updated with bug fixes and new features without affecting users.                                        |

**(b) Comparison of Prototype Model and Agile Model**

| <b>Prototype Model</b>                              | <b>Agile Model</b>                                             |
|-----------------------------------------------------|----------------------------------------------------------------|
| Focuses on developing an initial prototype.         | Focuses on developing the complete system in small iterations. |
| Used when requirements are unclear.                 | Used when requirements change frequently.                      |
| Customer provides feedback mainly on the prototype. | Customer provides feedback after every iteration.              |
| Prototype may be discarded or improved.             | Every iteration produces a working version of the software.    |
| Suitable for understanding user needs.              | Suitable for continuous development and frequent updates.      |

**(c) Recommended SDLC Model with Justification**

Recommended Model: Agile Model

**Justification:**

- Customer requirements change regularly.
- Agile supports frequent requirement changes without restarting the project.
- Continuous customer feedback ensures the system meets user expectations.
- New features can be added in short development cycles (sprints).



- Testing is performed in every iteration, improving software quality.
- It delivers working software quickly and allows continuous improvement.

### Final Answer:

For the **Smart Agriculture Monitoring System**, the **Agile Model** is the most suitable SDLC model because it supports **frequent requirement changes, continuous customer feedback, iterative development**, and **rapid delivery of working software**. This makes the system flexible, reliable, and easy to maintain as customer needs evolve.

5. Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

### Answer:

(a) Importance of UML in Object-Oriented Software Development

| Importance                   | Explanation                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Visual Representation</b> | UML provides graphical models to represent the structure and behavior of the software system. |
| <b>Better Communication</b>  | Helps developers, analysts, designers, and customers understand the system clearly.           |
| <b>System Design</b>         | Acts as a blueprint before coding begins.                                                     |
| <b>Error Reduction</b>       | Identifies design problems early, reducing development cost and time.                         |
| <b>Documentation</b>         | Provides proper documentation for future maintenance and enhancements.                        |

(b) UML Diagrams

| UML Diagram          | Description                                                                                     | Example (Smart Transport Management System)                                   |
|----------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>State Diagram</b> | Shows the different states of an object and the transitions between them during its life cycle. | A <b>Bus</b> changes states: Available → Assigned → In Transit → Maintenance. |

| UML Diagram               | Description                                                                            | Example (Smart Transport Management System)                                                     |
|---------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Component Diagram</b>  | Shows the organization and relationship between software components.                   | Components include <b>User Module, Booking Module, Payment Module, and Notification Module.</b> |
| <b>Deployment Diagram</b> | Shows the physical deployment of software on hardware devices and network connections. | Mobile App → Web Server → Database Server → GPS Tracking Device.                                |
| <b>Package Diagram</b>    | Organizes related classes into packages to improve modularity and readability.         | Packages such as <b>User Management, Vehicle Management, Booking, and Payment.</b>              |

### (c) How UML Models Improve Communication Among Stakeholders

| Stakeholder             | Benefit of UML Models                                               |
|-------------------------|---------------------------------------------------------------------|
| <b>Customers</b>        | Easily understand the system functionality through diagrams.        |
| <b>Developers</b>       | Use UML as a blueprint for coding the system correctly.             |
| <b>System Analysts</b>  | Clearly document and communicate system requirements.               |
| <b>Project Managers</b> | Monitor project progress and ensure the design meets project goals. |
| <b>Maintenance Team</b> | Understand the existing system for future updates and bug fixes.    |

### Final Answer:

UML (Unified Modeling Language) is an important modeling tool in **Object-Oriented Software Development** because it provides a clear visual representation of the system. Diagrams such as **State Diagram, Component Diagram, Deployment Diagram, and Package Diagram** help in designing, documenting, and understanding the **Smart Transport Management System**. UML improves communication among customers, developers, analysts, managers, and maintenance teams, resulting in **better software quality, easier maintenance, and efficient development**.

### 6. Consider developing a Smart Inventory Management System.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability.

Answer:

**(a) Concept of Modular Software Design**

| Concept                        | Explanation                                                                                                                                                                                             | Example (Smart Inventory Management System)                                                                                         |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Modular Software Design</b> | Modular software design is the process of dividing a software system into small, independent modules. Each module performs a specific function and can be developed, tested, and maintained separately. | Separate modules for <b>Inventory Management, Product Management, Supplier Management, Order Processing, and Report Generation.</b> |

**(b) Comparison of Procedural Programming and Object-Oriented Programming**

| Procedural Programming                           | Object-Oriented Programming (OOP)                               |
|--------------------------------------------------|-----------------------------------------------------------------|
| Focuses on functions or procedures.              | Focuses on objects and classes.                                 |
| Data and functions are separate.                 | Data and methods are combined in objects (encapsulation).       |
| Less secure because data is globally accessible. | More secure because data is hidden using encapsulation.         |
| Code reusability is limited.                     | Supports code reusability through inheritance and polymorphism. |
| Suitable for small applications.                 | Suitable for large, complex, and scalable applications.         |

**(c) How Modularity Improves Software Quality**

| Feature                 | Benefit in Smart Inventory Management System                                                                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Scalability</b>      | New modules such as <b>Barcode Scanning</b> or <b>Warehouse Management</b> can be added without affecting existing modules. |
| <b>Maintainability</b>  | Bugs can be fixed in one module without changing the entire system.                                                         |
| <b>Reusability</b>      | Modules like <b>User Login</b> and <b>Report Generation</b> can be reused in other software projects.                       |
| <b>Testing</b>          | Each module can be tested independently, making testing easier and more reliable.                                           |
| <b>Team Development</b> | Different developers can work on different modules simultaneously, improving development speed.                             |

**Final Answer:**

The **Smart Inventory Management System** uses **modular software design** by dividing the system into independent modules. Compared to **procedural programming, object-oriented**

OOAD

🎉 Kudos, keep up the motivation, RAGUL!

10470

Score

+185

Coins



Shop

Accuracy ⓘ



Performance Stats

45 questions

✓ 12 Correct

✓ 1 Partially correct

✗ 5 Incorrect

⌚ 26 s time/question

🔍 27 Unattempted

Review Questions

Click on the questions to see answers

### **RUBRICS FOR CALCULATION**

| Criteria                  | Weightage | Level 4 – Exemplary                                         | Level 3 – Proficient                    | Level 2 – Developing                         | Level 1 – Beginning                      |
|---------------------------|-----------|-------------------------------------------------------------|-----------------------------------------|----------------------------------------------|------------------------------------------|
| Understanding of Concepts | 25%       | Demonstrates thorough, accurate understanding of concepts   | Good understanding with minor gaps      | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation               | 25%       | Detailed, well-explained answers with relevant examples     | Clear explanation with adequate details | Limited explanation; lacks depth             | Poor or unclear explanation              |
| Application               | 20%       | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors      | Limited application with weak analysis       | No application or incorrect analysis     |
| Organization              | 20%       | Well-structured answers with logical flow and coherence     | Organized with minor issues in flow     | Basic structure, lacks clarity               | Poor organization, incoherent answers    |
| Accuracy                  | 10%       | Completely accurate and covers all required points          | Mostly accurate with minor omissions    | Partially correct with missing points        | Incorrect answers with major gaps        |